Team Details

Team: sssharihar an82 Team Leader: Hariharan S Email: hariharan.s@stjosephs.edu.in Hackathon: FutureStack

Completed Features

Key point: end-to-end brand-review workflow in a static web app. - Company login with demo codes (Mozilla, Mailchimp, Acme) - Draft upload via paste, .txt, .md, or PDF (selectable text) - Guidelines preview for the selected company - Website destination selector (Website, LinkedIn, Instagram, X, Facebook) - Deterministic brand-rule scan against loaded guideline pack - Findings highlighted directly in the original draft - Mistake list below the draft with rule explanation and suggested fix - One-click apply suggestion to generate revised draft - Downloadable audit report (.txt) - Logo quality checks (dimensions, transparency, clear-space proxy) - Optional AI polish via Groq/OpenAI-compatible API (BYO key) - Profile & settings page (AI key/provider/model, current company) - Review history for the current browser session - Fully client-side — no backend, no account creation, no data leaves the browser unless AI polish is used

Technology and Implementation Details

Key point: fully client-side static app with deterministic rules and optional external AI polish. Frontend: plain HTML/CSS/JS (no framework), served as a static site via GitHub Pages. PDF handling: Mozilla PDF.js for text extraction from selectable PDFs. Rule engine: deterministic regular expressions loaded from JSON rule packs. Each rule has an id, description, and replacement text. The engine matches phrases in the draft, highlights them, and offers a contextual replacement. This keeps the core review predictable and offline-capable. Logo checks: canvas-based sampling of the uploaded image for dimensions and approximate transparency/clear space. These are file-quality proxies, not brand-approval checks. AI polish (optional): calls an OpenAI-compatible chat-completions endpoint (default Groq) with the revised draft and matched rule context. API key, base URL, and model are stored in browser localStorage. Output is labeled AI-generated and never silently replaces the reviewed copy. Data flow: draft + optional logo → in-browser processing → rule matches → highlighted original draft + findings panel → revised draft → report export. Nothing is stored server-side. Hard bits I actually built: getting PDF.js to extract text without a server, building a deterministic rule engine that produces contextual replacements instead of dumb string substitution, and wiring the AI polish to work with any OpenAI-compatible provider while keeping the key in localStorage.

What Changed From the Original Proposal, and Why?

Key point: scope reduced for a reliable 48-hour build; roadmap unchanged. - Dropped the Tauri v2 desktop wrapper. Packaging Rust + React added too much build friction. I switched to a static web app that runs in any browser. The same logic ports to Tauri later without changing the rule engine or AI layer. - Dropped the local Ollama/Llama model plan. Running a quantized 8B model on an 8 GB laptop with no GPU was unreliable and slowed the UI. I replaced it with an optional BYO-key AI polish that uses Groq or any OpenAI-compatible provider. The deterministic rules remain the source of truth; AI is just a rewrite helper. - Added an in-browser logo-check tab. The original proposal mentioned logo review, but the Round 1 plan did not specify implementation details. I added basic file-level checks as a placeholder for the proper approved-asset matching planned for Round 3. - Kept the rule packs as JSON files rather than a database. This makes the prototype auditable and easy to extend, and maps directly to the sqlite-vec + document store planned for Round 3.

What Does NOT Work Yet?

Key point: core review works without a key; AI, auth, OCR, and logo matching are incomplete. - AI polish is off by default. You need to paste a Groq/OpenAI key in Profile & settings for it to work. Without a key, the deterministic scan still runs. - Logo checks are preliminary. They look at file properties, not whether the logo is the approved brand asset. That needs asset hashing + reference matching. - PDF support is text-only. Scanned PDFs need OCR, which is planned for Round 3. - Authentication is demo-only. The codes are shared and do not isolate company data. Real auth + workspaces are Round 3 work. - The AI rewrite does not re-run deterministic checks automatically. Users should re-analyze the AI output before accepting it.

Three Improvements for Round 3

Improvement 1: Multimodal Document Analysis with OCR

What changes: scanned PDFs, images, and slide decks become reviewable, not just selectable text. Add OCR and multimodal document analysis so the tool can review scanned PDFs, images, and slide decks — not just selectable text. Page-level or region-level evidence links each finding back to the exact page or area of the document.

Improvement 2: Enterprise Governance and Audit

What changes: real access control and traceable decisions, instead of shared demo codes. Introduce real enterprise governance: actual authentication, role-based permissions, isolated company workspaces, versioned rule packs, and an auditable approval history. Unauthorized users cannot access another company's workspace, and reviewer decisions are traceable.

Improvement 3: Controlled Publishing and Quality Measurement

What changes: publishing becomes approval-gated and measurable, instead of only suggesting fixes. Build controlled publishing integrations and quality measurement: scoped channel integrations (CMS, social schedulers), approval-before-publishing controls, audit trails, and false positive/override metrics so teams can measure how well the tool works over time.

Live Links

Working product: https://hariharan-007design.github.io/Rakshak2.0/ Source code: https://github.com/hariharan-007design/Rakshak2.0.git